# A SIMPLE PROOF OF P v NP

Christopher Robinson Tompkins III, Independent, Richmond, VA
Corresponding author. Email: chtompki@apache.org,
chtompki@vt.edu, tompkinscr@vcu.edu

**A proof of the PvNP problem such that a grade schooler should be able to understand.**
**I am writing for my daughters Margo (10) and Nora (12).**

**(0)**

*Dedicated to the memory of those killed at Virginia Tech,*
*and in solidarity with those who were injured, their families,*
*and the Virginia Tech community.*

*I would like to acknowledge the following people that helped me along the way: mom, dad, Margo, Nora, Griff Elder (and the 2007 VT Mathematics department and all my friends from there), George McNulty (and the 2009 USC Mathematics department and all my friends from there), Andy Lewis (and the VCU 2026 Mathematics Department and all my friends from there), and those at NASA whom I've worked with.*

**(1)**

## DEFINITIONS

Before going into the proof we need to clarify what the complexity classes **P** and **NP** are. Note we could go into extensive details about *deterministic Turing machines* and more importantly *non-deterministic Turing machines* (more specifically Turing machines that are *not* deterministic), but we need not add that amount of complexity for a paper that needs to be understood by grade schoolers.

We mainly concern ourselves with the English verbal definitions here, as we need no more than that for our proof.

**P** is the *deterministic polynomial time algorithms*.

**NP** is the *non-deterministic polynomial time algorithms* or more importantly *the **not** deterministic polynomial time algorithms*.

**(2)**

## CLAIM AND PROOF

**Theorem.** ***P = NP** and **P ≠ NP**, simultaneously.*

*Proof.* We begin by writing out **P** and **NP** in the English as stipulated above with assuming they are equal. So we set:

$$\begin{aligned} \text{polynomial time algorithms} &= \text{non-deterministic polynomial time algorithms} \\ \text{(polynomial time algorithms)} &= \text{non-(deterministic polynomial time algorithms)} \\ \text{(polynomial time algorithms)} &= \text{not-(deterministic polynomial time algorithms)} \\ q &= \text{not } q, \end{aligned}$$

where $q$ = "polynomial time algorithms". But $q$ = not $q$ is simply the liar's paradox, otherwise known as *The Great Antinomy* in the *Principia Mathematica*. **QED** □

So the **P** versus **NP** problem is independent (i.e. is simultaneously *true* and *false*). A more complex combinatorial proof is that it reduces to the Continuum Hypothesis, an exercise left to the reader after the hit and needed set-theory correction (note: set-theory has strayed from the original definitions unfortunately) that $\aleph_0$ is the size of the *arbitrarily large finite integer n*, as given by Cantor and Russell.

# REFERENCES


1. G. Cantor, "Ueber eine elementare Frage der Mannigfaltigkeitslehre," *Jahresbericht der Deutschen Mathematiker-Vereinigung* **1**, 72–78 (1890/91).

2. Stephen Cook, "The complexity of theorem-proving procedures," *Conference Record of Third Annual ACM Symposium on Theory of Computing*, pages 151-158, 1971.

3. Stephen Cook, "The **P** versus **NP** Problem," *Official Problem Description for the Millennium Prize Problems*, Clay Mathematics Institute, 2000, `https://www.claymath.org/wp-content/uploads/2022/06/pvsnp.pdf`.

4. A. N. Whitehead and B. Russell, *Principia Mathematica*, vols. 1–3 (Cambridge University Press, 1910–1913).